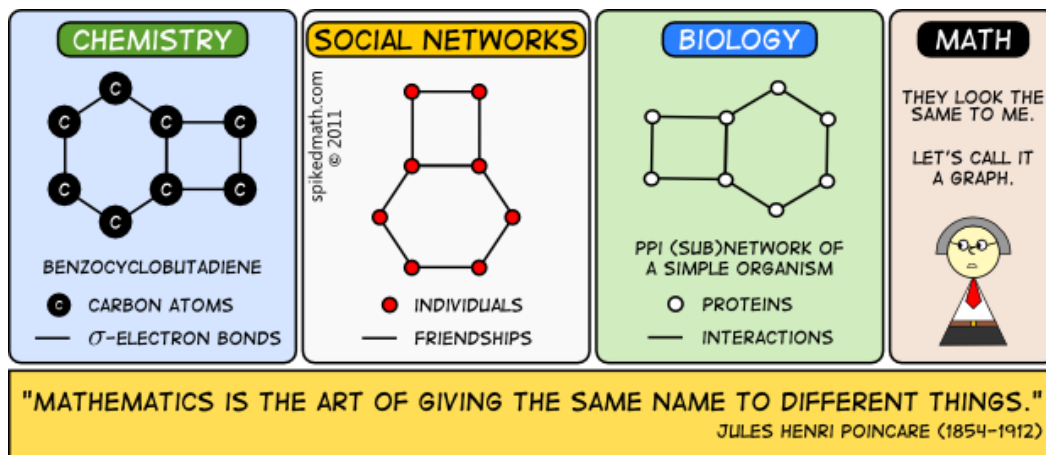


Structures de données relationnelles : les graphes

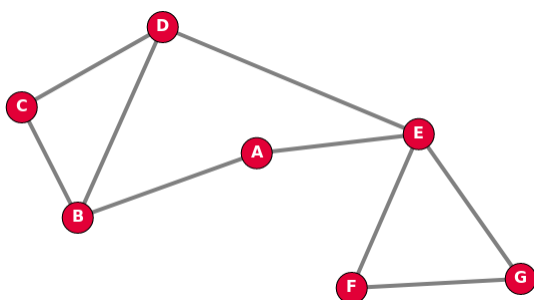
1. Notion de graphe et vocabulaire



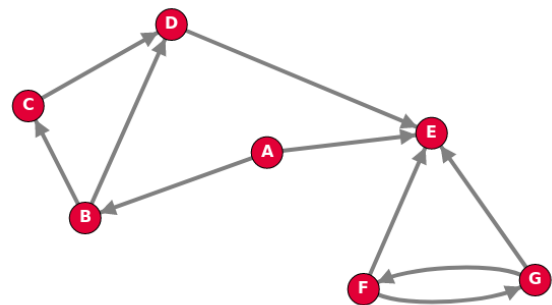
Le concept de graphe permet de résoudre de nombreux problèmes en mathématiques, comme en informatique. C'est un outil de représentation très courant. On les utilise, par exemple, pour représenter :

- ✚ Les relations entre les personnes d'un groupe ou d'un réseau social ;
- ✚ Un réseau routier, de bus, de métro ;
- ✚ Un réseau informatique ;
- ✚ Un circuit électrique.

Un graphe est un ensemble d'objets, appelés **sommets** ou parfois **nœuds** (vertex or nodes en anglais) reliés par des **arêtes** ou **arcs** (edge en anglais) selon que le graphe est **non orienté** ou **orienté**.



Exemple de graphe non orienté



Exemple de graphe orienté

L'**ordre** d'un graphe est le nombre de ses sommets : les deux graphes précédents sont d'**ordre 7**.

a. Graphes non orientés

Dans un graphe **non orienté**, les **arêtes** peuvent être empruntées dans les deux sens, et une **chaîne** est une suite de sommets reliés par des arêtes, comme, par exemple, C - B - A - E.

La **longueur** de cette chaîne est alors 3, soit le nombre d'arêtes.

Une chaîne dont le sommet de début et le sommet de fin sont identiques est appelé **cycle**, comme, par exemple B - A - E - D - C - B.

Les sommets B et E sont **adjacents** au sommet A : ce sont les **voisins** de A.

Le **degré** d'un sommet est le nombre d'arêtes ayant ce sommet pour extrémités, les boucles étant comptées deux fois. Un sommet de degré zéro est dit **isolé**.

Structures de données relationnelles : les graphes

b. Graphes orientés

Dans un graphe **orienté**, les **arcs** ne peuvent être empruntés que dans le sens de la flèche, et un **chemin** est une suite de sommets reliés par des arcs, comme, par exemple, $B \rightarrow C \rightarrow D \rightarrow E$.

Un chemin dont le sommet de début et le sommet de fin sont identiques est appelé **circuit** comme, par exemple, $B \rightarrow C \rightarrow D \rightarrow B$.

Les sommets C et D sont **adjacents** au sommet B (mais pas A !) : ce sont les **voisins** de B.

On appelle **degré** d'un sommet, le nombre d'arcs dont ce sommet est une extrémité.

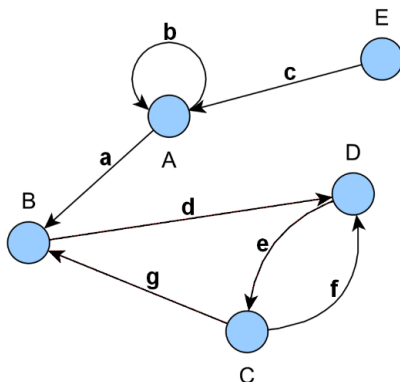
En général, on note d^+ le nombre d'arcs partant du sommet et d^- le nombre d'arcs pointant sur ce sommet : la somme des deux correspond au degré du sommet.

c. Graphes étiquetés et graphes pondérés

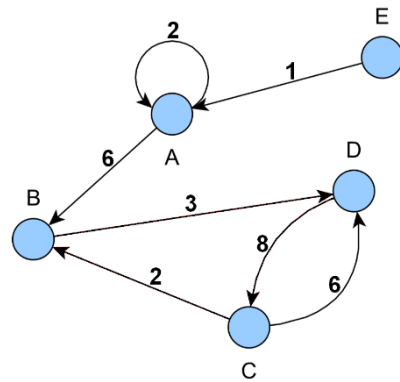
Un graphe **étiqueté** est un graphe où chaque relation (arc ou arête) est affectée d'un symbole.

Un graphe **pondéré** est un graphe où chaque relation (arc ou arête) est affectée d'une valeur numérique (souvent positive) appelé **poids** (parfois **mesure**, **coût** ou **valuation**).

Le **poids** d'une chaîne ou d'un chemin est la somme des poids des relations qui la composent.



Exemple de graphe étiqueté



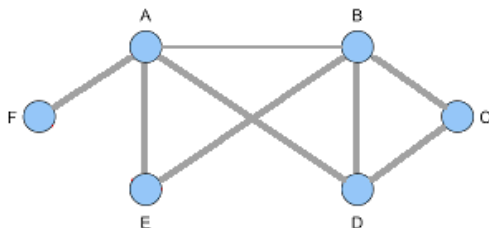
Exemple de graphe pondéré

En pratique :

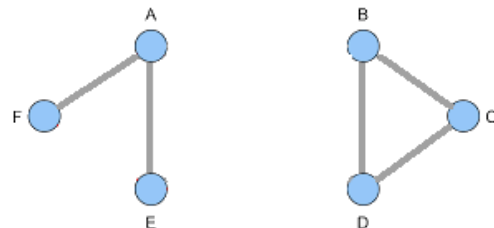
- dans le protocole de routage OSPF que nous verrons plus tard dans la partie sur les réseaux, on pondère les liaisons entre routeurs par le coût ;
- dans un réseau routier entre plusieurs villes, on pondère par les distances ou par le temps.

d. Graphes connexes et graphes complets

Un graphe non orienté est dit **connexe** si n'importe quelle paire de sommets peut toujours être reliée par une chaîne. Autrement dit, un graphe est connexe s'il est « en un seul morceau ».



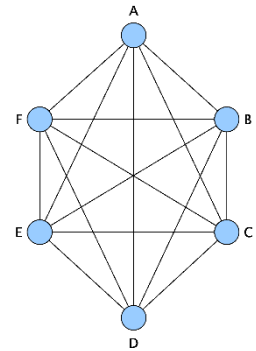
Exemple de graphe connexe



Exemple de graphe non connexe

Structures de données relationnelles : les graphes

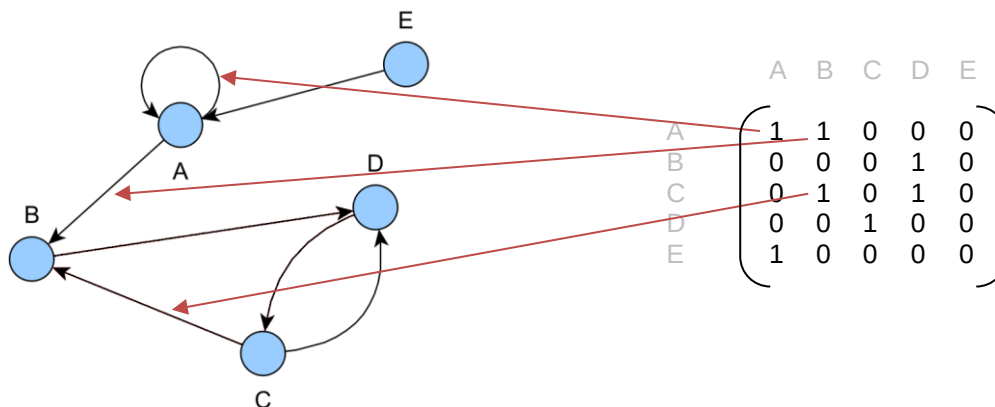
Un graphe non orienté est dit **complet** si chacun des sommets est relié directement à tous les autres sommets.



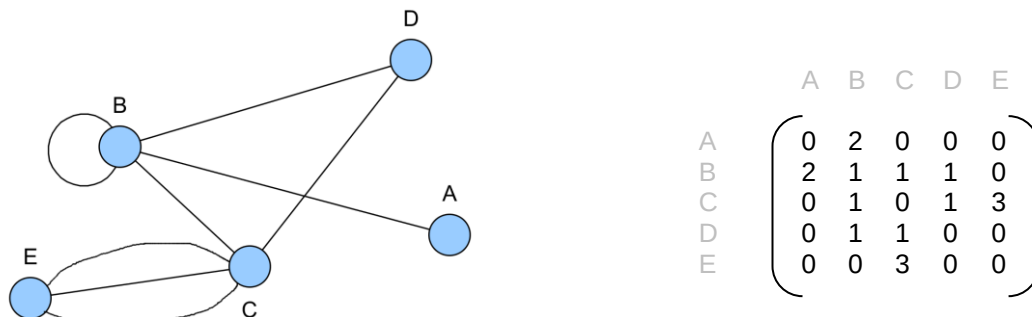
2. Représentations d'un graphe

a. Représentation par matrice d'adjacence

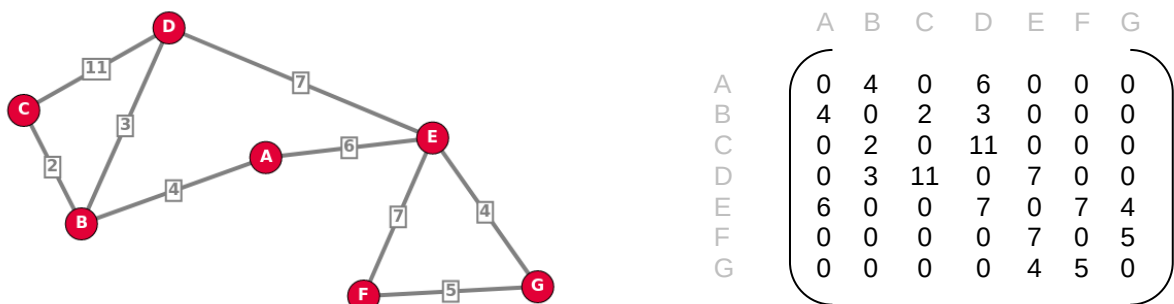
A un graphe, on associe un tableau à deux dimensions où chaque nombre à l'intersection entre une ligne i et une colonne j correspond au nombre de relations entre le sommet i et le sommet j .
Un tel tableau est appelé **matrice d'adjacence**.



Si le graphe est **non orienté**, la matrice est toujours **symétrique** (par rapport à la diagonale descendante).



Si le graphe est **pondéré**, on indique les poids des arêtes dans la matrice :



Structures de données relationnelles : les graphes

En python

Pour représenter **une matrice d'adjacence en python**, on peut utiliser une liste de listes, chaque sous-liste représente une ligne de la matrice. Par exemple,

$$\begin{pmatrix} 0 & 2 & 0 & 0 & 0 \\ 2 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 3 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \end{pmatrix} \Rightarrow \begin{matrix} [0, 2, 0, 0, 0], \\ [2, 1, 1, 1, 0], \\ [0, 1, 0, 1, 3], \\ [0, 1, 1, 0, 0], \\ [0, 0, 3, 0, 0] \end{matrix}$$

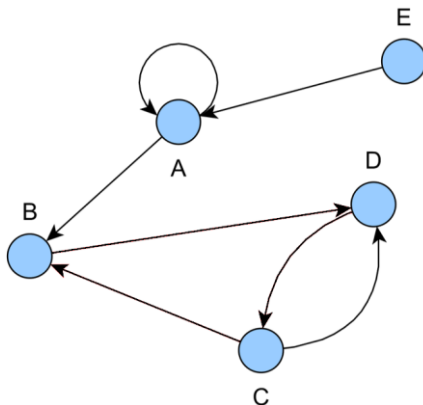
Remarques :

- Pour un graphe à n sommets, la complexité spatiale (place en mémoire) est en $O(n^2)$.
- Tester si un sommet est isolé (ou connaître ses voisins) est en $O(n)$ puisqu'il faut parcourir une ligne, mais tester si deux sommets sont adjacents (voisins) est en $O(1)$: c'est un simple accès au tableau.

b. Représentation par listes d'adjacence

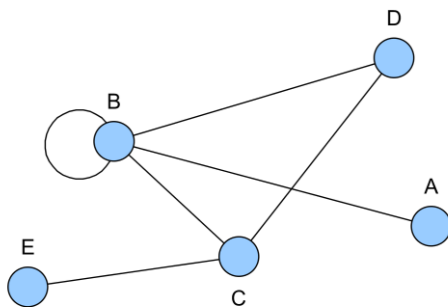
On peut aussi associer, à un graphe orienté :

- **une liste de successeurs** qui associe, à chacun des sommets, la liste des sommets que l'on peut atteindre directement par un arc ;
- **une liste de prédécesseurs** qui associe, à chacun des sommets, la liste des sommets qui permettent d'atteindre ce sommet directement par un arc.



Sommet	Liste successeurs	Liste prédécesseurs
A	A, B	A, E
B	D	A, C
C	B, D	D
D	C	B, C
E	A	$\emptyset$

On peut aussi associer à un graphe non orienté **la liste des voisins** qui correspond aux sommets que l'on peut atteindre directement par une arête.



Sommet	Liste des voisins
A	B
B	A, B, D, C
C	B, D, E
D	B, C
E	C

Structures de données relationnelles : les graphes

En python

Pour représenter une liste de successeurs, de prédécesseurs ou de voisins, on utilisera un dictionnaire.

Exemple d'un graphe non pondéré :

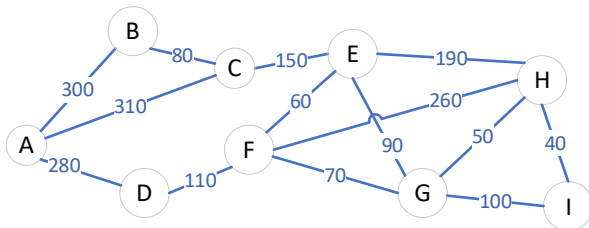
Sommet	Liste prédécesseurs
A	A, E
B	A, C
C	D
D	B, C
E	$\emptyset$



```
{ 'A' : [ 'A', 'E' ],  
  'B' : [ 'A', 'C' ],  
  'C' : [ 'D' ],  
  'D' : [ 'B', 'C' ],  
  'E' : [ ] }
```

Dans le cas d'un graphe **pondéré**, on pourra utiliser un dictionnaire de dictionnaires.

Exemple d'un graphe pondéré :



```
{ 'A' : { 'B' : 300, 'C' : 310, 'D' : 280 },  
  'B' : { 'A' : 300, 'C' : 80 },  
  'C' : { 'A' : 310, 'B' : 80, 'E' : 150 },  
  'D' : { 'A' : 280, 'F' : 110 },  
  'E' : { 'C' : 150, 'F' : 60, 'G' : 90, 'H' : 190 },  
  'F' : { 'D' : 110, 'E' : 60, 'G' : 70, 'H' : 260 },  
  'G' : { 'E' : 90, 'F' : 70, 'H' : 50, 'I' : 100 },  
  'H' : { 'E' : 190, 'F' : 260, 'G' : 50, 'I' : 40 },  
  'I' : { 'G' : 100, 'H' : 40 }  
}
```

Ou un dictionnaire de listes de tuples :

```
{ 'A' : [ ('B', 300), ('C', 310), ('D', 280) ],  
  'B' : [ ('A', 300), ('C', 80) ],  
  'C' : [ ('A', 310), ('B', 80), ('E', 150) ],  
  'D' : [ ('A', 280), ('F', 110) ],  
  'E' : [ ('C', 150), ('F', 60), ('G', 90), ('H', 190) ],  
  'F' : [ ('D', 110), ('E', 60), ('G', 70), ('H', 260) ],  
  'G' : [ ('E', 90), ('F', 70), ('H', 50), ('I', 100) ],  
  'H' : [ ('E', 190), ('F', 260), ('G', 50), ('I', 40) ],  
  'I' : [ ('G', 100), ('H', 40) ]  
}
```

Remarques :

- Pour un graphe à n sommets et m arêtes, la complexité spatiale (place en mémoire) est en $O(n + m)$: c'est beaucoup mieux qu'une matrice d'adjacence lorsque le graphe comporte peu d'arêtes (i.e. beaucoup de 0 dans la matrice, non stockés avec des listes).
- Tester si un sommet est isolé (ou connaître ses voisins) est en $O(1)$ puisqu'on y accède immédiatement, mais tester si deux sommets sont adjacents (voisins) est en $O(n)$ car il faut parcourir le dictionnaire ou la liste.

Structures de données relationnelles : les graphes

3. Opérations principales sur un graphe

Exemples :

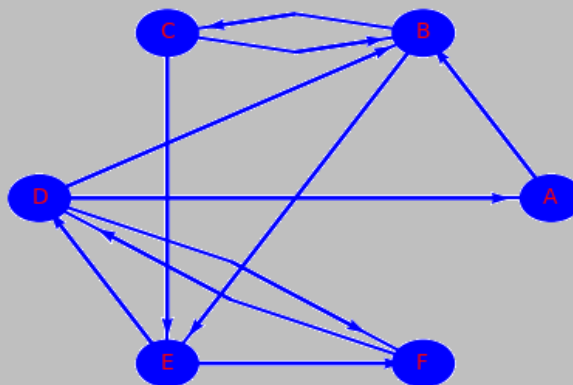
- ✚ CREER_GRAPHE_VIDE() qui retourne un graphe vide.
- ✚ AJOUTER_SOMMET(G, s) qui ajoute au graphe G le sommet s si et seulement si le sommet s n'existe pas.
- ✚ AJOUTER_ARC(G, sd, sf) qui ajoute un arc au graphe G entre le sommet de début sd et le sommet de fin sf si et seulement si sd et sf existent et que l'arc entre sd et sf n'existe pas.
- ✚ SUPPRIMER_SOMMET(G, s) qui supprime du graphe G le sommet s si et seulement si le sommet s existe.
- ✚ SUPPRIMER_ARC(G, sd, sf) qui supprime l'arc du graphe G entre sd et sf si et seulement si cet arc existe.
- ✚ AJOUTER_ARETE(G, sd, sf) qui ajoute une arête au graphe G entre le sommet de début sd et le sommet de fin sf si et seulement si sd et sf existent et que l'arête entre sd et sf n'existe pas.
- ✚ SUPPRIMER_ARETE(G, sd, sf) qui supprime l'arête du graphe G entre sd et sf si et seulement si cette arête existe.

4. Exercices

I) Dessiner le graphe décrit par le pseudo-code ci-dessous

```
G1 = CREER_GRAPHE_VIDE()
AJOUTER_SOMMET(G1, 'A')
AJOUTER_SOMMET(G1, 'B')
AJOUTER_SOMMET(G1, 'C')
AJOUTER_SOMMET(G1, 'D')
AJOUTER_SOMMET(G1, 'E')
AJOUTER_SOMMET(G1, 'F')
AJOUTER_ARC(G1, 'A', 'B')
AJOUTER_ARC(G1, 'B', 'C')
AJOUTER_ARC(G1, 'C', 'F')
AJOUTER_ARC(G1, 'F', 'E')
AJOUTER_ARC(G1, 'E', 'D')
```

II) Donner la matrice d'adjacence du graphe suivant :



III) Donner les listes successeurs et prédécesseurs du graphe ci-dessus (II).

IV) Donner le graphe dont la matrice d'adjacence est la suivante, vous préciserez s'il est orienté ou non.

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

Structures de données relationnelles : les graphes

V) Donner le (multi)graphe de la matrice d'adjacence suivante, vous préciserez s'il est orienté ou non.

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 \\ 3 & 1 & 0 & 0 & 1 \\ 1 & 0 & 4 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

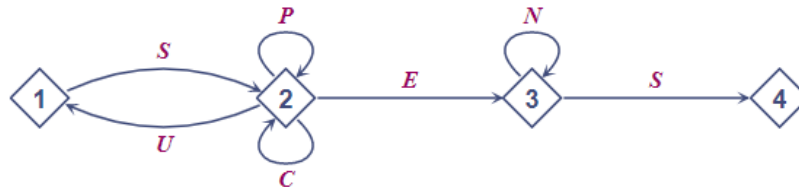
VI) Donner le graphe de ce réseau d'amis :

Utilisateur	Ami avec
A	B, D, G
B	A, C, G
C	B, F, G
D	A
E	F
F	C, E
G	A, B, C

VII) Donner le graphe associé à la liste des successeurs :

Sommet	Liste successeurs
A	D, F
B	A, E
C	B
D	B, C
E	A, D
F	E

VIII) Pour accéder à sa messagerie, Antoine a choisi un code qui doit être reconnu par le graphe étiqueté suivant, de sommets 1, 2, 3 et 4 :



Parmi les trois codes suivants, donner le (ou les) code(s) reconnu(s) par le graphe :
SUCCÈS ; SCENES ; SUSPENS.

Donner la matrice d'adjacence de ce graphe.

Codage d'un graphe en Python

Pour cela, nous allons utiliser des dictionnaires, dans un premier temps.
Créer, sous Spyder, un fichier nommé *graphes.py*.

I) Coder la fonction CREER_GRAPHE_VIDE() qui renvoie un dictionnaire vide lorsqu'on l'appelle.

II) Coder la fonction AJOUTER_SOMMET(G, S) d'après l'algorithme ci-dessous :

Si S est déjà une clé du dictionnaire

Alors :

afficher « Ce sommet existe déjà, impossible de l'ajouter »

Sinon :

ajouter la clé S avec comme valeur une liste vide

Renvoyer G

Structures de données relationnelles : les graphes

Tester en ajoutant les sommets puis afficher le graphe suivant :

```
G1 = CREER_GRAPHE_VIDE()
AJOUTER_SOMMET(G1, 'A')
AJOUTER_SOMMET(G1, 'B')
AJOUTER_SOMMET(G1, 'C')
AJOUTER_SOMMET(G1, 'D')
AJOUTER_SOMMET(G1, 'E')
AJOUTER_SOMMET(G1, 'F')
print(G1)
```

Faire une capture d'écran du résultat.

III) Coder la fonction `AJOUTER_ARC(G, sd, sf)` d'après l'algorithme ci-dessous :

Si `sd` est une clé du dictionnaire (le sommet existe) et `sf` est une clé du dictionnaire (le sommet existe) et si l'arc entre `sd` et `sf` n'existe pas encore (`sf not in G[sd]`)

Alors :

- ajouter `sf` dans la liste des valeurs du sommet `sd`
- renvoyer `G`

Sinon :

- afficher « Impossible de rajouter cet arc »

Tester en ajoutant les arcs après les sommets puis afficher le graphe suivant :

```
AJOUTER_ARC(G1, 'A', 'B')
AJOUTER_ARC(G1, 'B', 'C')
AJOUTER_ARC(G1, 'C', 'F')
AJOUTER_ARC(G1, 'F', 'E')
AJOUTER_ARC(G1, 'E', 'D')
print(G1)
```

Faire une capture d'écran du résultat et vérifier si cela correspond à la question I.

IV) Coder la fonction `AJOUTER_ARETE(G, sd, sf)` d'après l'algorithme ci-dessous :

Si `sd` est une clé du dictionnaire (le sommet existe) et `sf` est une clé du dictionnaire (le sommet existe) et si l'arête entre `sd` et `sf` n'existe pas encore

Alors :

- ajouter `sf` dans la liste des valeurs du sommet `sd`
- si le sommet `sd` est différent du sommet `sf`
- Alors :
 - ajouter `sd` dans la liste des valeurs du sommet `sf`
- renvoyer `G`

Sinon :

- afficher « Impossible de rajouter cette arête »

Tester en créant un graphe `G2` vide, puis en ajoutant les sommets `A`, `B`, `C`, `D`, puis les arêtes (`A-B`), (`A-C`), (`A-D`), (`B-C`), (`B-D`) et (`C-D`) et enfin afficher le graphe `G2`.

Faire une capture d'écran du résultat.

V) Coder la fonction `SUPPRIMER_SOMMET(G, S)` d'après l'algorithme ci-dessous :

Si `S` est une clé du dictionnaire

Alors :

- Supprimer la clé `S` à l'aide de la fonction `del` (instruction `del (G[S])`)
- Pour chaque clé du dictionnaire :
 - Si `S` est une valeur de la liste associée à la clé :
 - Alors :
 - `index_de_S = G[cle].index(S)`
 - Supprimer `S` dans la liste `G[cle]` à cet index à l'aide de la fonction `del`.
- Renvoyer `G`

Sinon :

- afficher « Impossible de supprimer ce sommet »

Tester en ajoutant :

```
SUPPRIMER_SOMMET(G2, 'D')
SUPPRIMER_SOMMET(G2, 'E')
print(G2)
```

Faire une capture d'écran du résultat.

Structures de données relationnelles : les graphes

VI) Coder la fonction SUPPRIMER_ARC(G, sd, sf) d'après l'algorithme ci-dessous :
Si sd est une clé du dictionnaire (le sommet existe) **et** si arc vers sf existe (sf in G[sd])
Alors :
 index_sf = index de sf dans la liste G[sd]
 Supprimer l'arc entre sd et sf à l'aide de la fonction del
 renvoyer G
Sinon :
 afficher « Impossible de supprimer cet arc »

Tester en ajoutant :
SUPPRIMER_ARC(G1, 'A', 'B')
SUPPRIMER_ARC(G1, 'E', 'A')
print(G1)
Faire une capture d'écran du résultat.

VII) Coder la fonction SUPPRIMER_ARETE(G, sd, sf) d'après l'algorithme ci-dessous :
Si sd est une clé du dictionnaire (le sommet existe) **et** si l'arête vers sf existe (sf in G[sd])
Alors :
 index_sf = index de sf dans la liste G[sd]
 Supprimer l'arête entre sd et sf à l'aide de la fonction del
 Si les sommets sd et sf sont différents
 Alors :
 index_sd = index de sd dans la liste G[sf]
 Supprimer l'arête entre sf et sd à l'aide de la fonction del
 renvoyer G
Sinon :
 afficher « Impossible de supprimer cette arête »

Tester en ajoutant :
SUPPRIMER_ARETE(G2, 'A', 'B')
SUPPRIMER_ARETE(G2, 'E', 'A')
print(G2)
Faire une capture d'écran du résultat.

VIII) Coder la fonction nb_sommets d'après l'algorithme ci-dessous :
Renvoyer la longueur de G

Ttester en ajoutant :
print("Le nombre de sommets de G1 est : " + str(nb_sommets(G1)))
print("Le nombre de sommets de G2 est : " + str(nb_sommets(G2)))
Faire une capture d'écran du résultat.

IX) Coder la fonction deg_non_oriente(G, s) , cela correspond au nombre de valeurs dans la liste du sommet S d'après l'algorithme ci-dessous :

Affecter la valeur 0 à la variable nb_boucles
Si la liste associée à la clé s n'est pas vide
Alors :
 Pour chaque clé du dictionnaire
 Si la clé appartient aussi à la liste associée à cette clé
 Augmenter de 1 la variable nb_boucles
Renvoyer la somme de nb_boucles et de la taille de la liste associée au sommet s.

Tester en ajoutant :
print("Le degré du sommet A de G2 est : " + str(deg_non_oriente(G2, 'A')))
print("Le degré du sommet C de G2 est : " + str(deg_non_oriente(G2, 'C')))
Faire une capture d'écran du résultat.

Structures de données relationnelles : les graphes

X) Coder la fonction `deg_oriente(G, s)` (se sera d+ additionné à d-) d'après l'algorithme ci-dessous :

Affecter 0 à la variable `deg`

Si la liste de la clé `s` n'est pas vide

Alors :

`deg` = taille de la liste du sommet `s` (cela correspond à d+ : arcs sortant de `s`)

Pour chaque sommet du dictionnaire (chaque clé du dictionnaire) :

Si `s` est dans la liste des valeurs de ce sommet

Alors : on ajoute 1 à `deg`

Renvoyer `deg`

Tester en ajoutant :

```
print("Le degré du sommet A de G1 est : " + str(deg_oriente(G1, 'A')))
```

```
print("Le degré du sommet B de G1 est : " + str(deg_oriente(G1, 'B')))
```

Faire une capture d'écran du résultat.

XI) Soit l'algorithme ci-dessous :

Affecter à la variable `n`, le nombre de sommets de `G` à l'aide de la fonction `nb_sommets`

Affecter une liste vide à la variable `liste_sommets` (celle-ci stockera les sommets dans l'ordre alphabétique)

Créer, par compréhension, la variable `mat` qui est une matrice d'ordre `n` composée uniquement de 0

Pour chaque clé du dictionnaire `G` trié (`sorted(G.keys())`) :

Ajouter clé dans `liste_sommets`.

Pour chaque clé du dictionnaire `G` :

Créer la liste `valeurs` = la liste de la clé de `G` (`valeurs = G[cle]`)

Trier cette liste à l'aide de la méthode `sort`

Créer la variable `index_ligne` = index du sommet clé (`liste_sommets.index(cle)` qui donne la ligne de la matrice `mat` associée au sommet)

Pour `sommet` dans `valeurs` :

Créer la variable `index_colonne` = index de sommet dans la liste `liste_sommets`

Mettre à 1 l'élément `mat[index_ligne][index_colonne]`

Renvoyer `mat`

Tester cet algorithme à la main à l'aide du graphe : {'A': ['B', 'D'], 'B': ['A', 'C'], 'C': ['B'], 'D': ['A', 'B']}

Coder la fonction `conversion_g_en_mat_adj(G)` à l'aide de l'algorithme ci-dessus

Tester pour vérifier votre résultat précédent. On nommera G3 ce graphe.

Vérifier aussi votre programme avec le graphe de la question II), on nommera G4 ce graphe.

Faire une capture d'écran des résultats.

XII) Coder la fonction `conversion_mat_adj_en_g(mat)` d'après l'algorithme ci-dessous :

Affecter à la variable `graphe` un graphe vide à l'aide de la fonction `CREER_GRAPHE_VIDE()`

Pour `i` allant de 0 à la taille de `mat` :

Ajouter, à l'aide de la fonction `AJOUTER_SOMMET` codée plus haut, le sommet (A pour `i = 0`, B pour `i = 1` : on utilisera la fonction `chr` qui renvoie le caractère de la table ASCII dont le code, en décimal, est donné en argument, en sachant que la valeur décimale est 65 pour A, 66 pour B....).

Pour `k` allant de 0 à la taille de `mat` :

Pour `j` allant de 0 à la taille de `mat` :

Si `mat[k][j] == 1` (cela signifie qu'il y a un arc) :

Alors :

Ajouter l'arc dans le graphe à l'aide de la fonction `AJOUTER_ARC` codée précédemment (pour définir `sd` et `sf` on utilisera à nouveau le code ASCII)

Renvoyer `graphe`.

Tester avec la matrice de la question IV, on nommera ce graphe G5.

Faire une capture d'écran du résultat.

Sources :

- [Cours de Terminale NSI de Cédrix Gouygou](#)
- [Cours de Terminale NSI de Gisèle Bareux](#)
- [Cours de Terminale NSI de Grégory Viateau](#)